

Local development

Developer reference: running locally with no Google account or Gemini key, plus inspection SQL.

Quick start

```
make setup && make demo
```

- Runs Postgres in Docker via Supabase, with a fake Google client.

Prerequisites

- Docker (running), Python 3.11+, Supabase CLI (global install). `make setup` checks all three.

```
brew install supabase/tap/supabase
npm install -g supabase
```

Windows

- `make` targets live in `tasks.py` (standard library only): `python tasks.py doctor, setup, demo, test`.
- Activate the venv with `.venv\Scripts\Activate.ps1`; if blocked, `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`.
- No `requirements.txt`. Manual install: `pip install -e ".[dev]"` (quotes required in PowerShell).
- Supabase CLI: `scoop bucket add supabase https://github.com/supabase/scoop-bucket.git`, then `scoop install supabase`.

Database

```
make db-up
make db-reset
make db-down
```

- Postgres: `postgresql://postgres:postgres@localhost:64322/postgres` (set as `CUFA_DATABASE_URL`). Studio: `http://localhost:64323`. API: `http://localhost:64321`.
- **`make db-reset` destroys all local data.** It applies `supabase/migrations/*.sql`, then `supabase/seed.sql` (cohorts `cu-2026` and `demo` only). Load fellows and sessions with `cufa load-roster / cufa load-sessions`.
- `google_credential.refresh_token_enc` should render as bytes; anything readable means encryption was bypassed.

Environment

```
cp .env.example .env
.venv/bin/python -m cufa.crypto keygen
```

- Key goes in `CUFA_ENCRYPTION_KEY`.

Variable	Effect
<code>CUFA_DATABASE_URL</code>	Postgres DSN
<code>CUFA_FAKE_GOOGLE=1</code>	Use <code>FakeGoogleClient</code> , no Google calls
<code>CUFA_FAKE_GOOGLE_STATE</code>	Fake state file (<code>fixtures/fake_google_state.json</code>)
<code>CUFA_ENCRYPTION_KEY</code>	Required to store the refresh token
<code>GEMINI_API_KEY</code>	Enables tier 2; absent gives <code>rule_name='ai_unavailable'</code>
<code>CUFA_MAX_EDIT_DISTANCE</code>	Passphrase tolerance, default 1
<code>CUFA_LOG_LEVEL</code>	Only <code>DEBUG</code> logs raw emails

Make targets

- Overrides: `COHORT`, `FIXTURES`, `SHEET_TZ`, `PORT`. Every recipe sets `CUFA_FAKE_GOOGLE=1`.
- `make demo`: resets, then runs both parts on synthetic data.
- **`make demo` refuses on a database that looks real.** Use `CUFA_DATABASE_URL=.../cufa_demo make demo` or `CUFA_DEMO_FORCE=1`.
- `make demo-again`: re-runs without reset; zero new rows.
- `make demo-ai`: adds live tier 2 and themes; needs `GEMINI_API_KEY`.
- `make demo-console`: demo, then `cufa serve` on `PORT`.
- `make test`: `pytest`, no network. `make clean`: `supabase stop --no-backup`, removes fixtures.

Inspecting results

- Paste into Studio or `psql`; replace `'demo'` with your cohort id.
- `fellow_id IS NULL` means "not on the roster" (identity resolves at read time).
- **`help_request` is absent from every view and snippet. Don't add a join to it** (see `docs/safeguarding.md`).

Current decisions per fellow

```
select f.fellow_id,
       f.full_name,
       count(v.checkin_id)           as checkins,
       count(*) filter (where v.status = 'attended')    as attended,
       count(*) filter (where v.status = 'needs_review') as needs_review,
       count(*) filter (where v.status = 'not_attended') as not_attended
from fellow f
left join v_checkin_resolved v on v.fellow_id = f.fellow_id
where f.cohort_id = 'demo'
group by f.fellow_id, f.full_name
order by f.full_name;
```

All check-ins with live decisions:

```

select coalesce(v.full_name, '(not on roster)')      as fellow,
       v.submitted_email,
       coalesce(v.session_title, '(no session matched)') as session,
       v.submitted_at_utc,
       v.passphrase_match,
       v.status,
       v.decided_by,
       coalesce(v.rule_name, v.ai_model, v.human_email) as decided_how,
       v.confidence,
       v.latency_seconds
from v_checkin_resolved v
where v.cohort_id = 'demo'
order by fellow, v.submitted_at_utc;

```

- With no `announced_at_utc`, the first submitter's `latency_seconds` reads 0.

Needs review

```

select v.checkin_id,
       v.submitted_at_utc,
       coalesce(v.full_name, '(not on roster)')      as fellow,
       v.submitted_email,
       coalesce(v.session_title, '(no session matched)') as session,
       v.passphrase_match,
       v.passphrase_raw                               as typed,
       coalesce(v.rule_name, v.ai_reasoning, '(no reason recorded)') as why
from v_checkin_resolved v
where v.status = 'needs_review'
order by v.submitted_at_utc;

```

- `needs_review` means undecided, never absent.

Check-ins never adjudicated:

```

select v.checkin_id, v.submitted_at_utc, v.submitted_email,
       v.session_match, v.passphrase_match
from v_checkin_resolved v
where v.decision_id is null
order by v.submitted_at_utc;

```

Unresolved addresses:

```

select iu.cohort_id, iu.email, iu.occurrence_count, iu.first_seen_at, iu.last_seen_at
from identity_unresolved iu
where iu.resolved_at is null
order by iu.last_seen_at desc;

```

AI decisions

```
select v.checkin_id,
       coalesce(v.session_title, '(no session matched)') as session,
       v.passphrase_raw as typed,
       v.status,
       v.confidence,
       v.ai_model,
       v.ai_prompt_version,
       v.ai_reasoning,
       v.decided_at
  from v_checkin_resolved v
 where v.decided_by = 'ai'
 order by v.decided_at desc;
```

- CLI: `cufa review --status ai --cohort demo`.

AI cache

```
select model, prompt_version, count(*) as cached_pairs,
       count(*) filter (where verdict) as verdict_true,
       min(created_at) as first_cached, max(created_at) as last_cached
  from ai_adjudication_cache
 group by model, prompt_version
 order by model, prompt_version;
```

Approximate hit rate:

```
with sent as (select count(*)::numeric as pairs
                from ai_adjudication_cache),
     judged as (select count(*)::numeric as decisions
                from attendance_decision
                where decided_by = 'ai')
select judged.decisions as ai_decision_rows,
       sent.pairs      as distinct_pairs_sent,
       round(100 * (1 - sent.pairs / nullif(judged.decisions, 0)), 1)
         as approx_cache_hit_pct
  from sent, judged;
```

- Exact numbers: `ai_calls=` and `cache_hits=` from `cufa adjudicate`.

Part A and Part B per session

```
select s.week_index,
       s.title,
       (select count(*) from checkin c where c.session_id = s.session_id) as part_a,
       (select count(*) from checkin_b b where b.session_id = s.session_id) as part_b
  from "session" s
 where s.cohort_id = 'demo'
 order by s.scheduled_at_utc;
```

Confidence by week

```
select week_index, session_title, responses, median, q1, q3, iqr
  from v_confidence_trend
 where cohort_id = 'demo'
 order by week_index nulls last;
```

Rejected confidence values

```
select submitted_email,
       extra_fields ->> '_confidence_rejected_raw' as raw_value
  from checkin_b
 where extra_fields ? '_confidence_rejected_raw';
```

What each form asked

```
select s.week_index, m.slot, m.rotating_kind, m.question_text
  from form_question_map m
 join session_form sf on sf.form_id = m.form_id and sf.part = 'b'
 join "session" s      on s.session_id = sf.session_id
 where s.cohort_id = 'demo' and m.slot = 'rotating'
 order by s.week_index;
```

Shoutouts waiting

```
select raw_text, session_title, submitted_at_utc
  from v_shoutout_review
 where cohort_id = 'demo'
 order by created_at;
```

Straight-lining

```
select full_name, confidence_raw, run_length, session_titles
  from v_confidence_straightline
 where cohort_id = 'demo'
 order by run_length desc;
```

Muddiest-point themes

```
select t.label, t.summary, b.rotating_text
  from muddiest_theme t
 join muddiest_theme_member m on m.theme_id = t.theme_id
 join checkin_b b            on b.checkin_b_id = m.checkin_b_id
 join "session" s            on s.session_id = t.session_id
 where s.cohort_id = 'demo' and t.superseded_at is null
 order by t.label, b.submitted_at_utc;
```

Provisioning log

```
select p.at,  
       coalesce(s.title, '(session deleted)') as session,  
       p.action,  
       p.outcome,  
       p.error,  
       p.request_summary  
from provisioning_log p  
left join "session" s on s.session_id = p.session_id  
order by p.at desc  
limit 50;
```

Failures only:

```
select p.at, p.action, p.outcome, p.error  
from provisioning_log p  
where p.outcome = 'failure'  
order by p.at desc;
```

Form readiness per session:

```
select s.title,  
       s.scheduled_at_utc,  
       sf.form_id,  
       sf.published_at,  
       sf.publish_verified_at,  
       sf.response_watermark,  
       sf.last_polled_at  
from "session" s  
left join session_form sf on sf.session_id = s.session_id  
where s.cohort_id = 'demo'  
order by s.scheduled_at_utc;
```

- `publish_verified_at` **NULL means the form accepts nothing** despite `published_at` (trap 1).

Troubleshooting

- `supabase start` fails: check `docker ps`; free ports 64321-64329 or edit `supabase/config.toml`.
- `connection refused` on 64322: run `python tasks.py db-up`.
- Schema stale: `supabase db reset` (migrations apply only on reset).
- "blocked, as designed" in `make demo` step 2 is expected.
- Tests must not hit the network: inject `google.factory.set_fake_client()` and `adjudicator=`.
- `logging_setup.RedactionFilter` redacts emails above DEBUG and credentials at every level.

See also

- `docs/setup/google-cloud.md`
- `docs/setup/console.md`
- `docs/google-api-traps.md`

- docs/decisions.md